淘宝主播Agent的Harness工程实战

直播技术团队 大淘宝技术 2026年8月6日 20:58 浙江 23人

本文介绍了淘宝主播Agent的Harness工程实战，旨在通过构建模型外层的工程化“骨架”解决大模型在高风险、高并发直播场景中不可控、易漂移的问题。文章首先定义了由执行循环、工具注册、上下文管理、状态存储、生命周期钩子和评估接口组成的Harness六元组，并确立了框架层兜底安全与状态、业务层专注技能开发的分层架构，采用MySQL、Hologres和GitLab分别存储会话、记忆和技能以实现逻辑统一与物理分治。在具体实践中，通过分层压缩与Reducer模式优化上下文，利用Schema强约束、幂等设计及五层纵深防御体系保障工具调用安全，引入沙箱隔离执行风险，并以DAG全局规划替代ReAct单步决策以提升长程任务的恢复性与效率。此外，文章详细阐述了基于会话、事实、行为三层结构的记忆体系，通过记忆对账机制与信任度演化算法，实现了Agent对主播偏好的精准理解与自适应输出，最终将不确定的模型能力转化为可用、可控、可演化的工业级产品。

01 为什么主播 Agent 需要Harness?

OpenClaw、Claude Code、Hermes 这类智能体产品把 Harness Engineering 这个词带火了。它的核心主张很简单：模型能力是概率的、会漂移的、偶尔会失控的，真正让 Agent 可用、可控、可演化的，是模型外面那一层工程化的"骨架"（Harness）：结构化的上下文、约束性的工具协议、生命周期的钩子、可恢复的状态、可观测的评估。但大部分公开的 Harness 实践，验证场景都是"个人助手"形态：单用户、本机运行、出了错自己看日志重跑一次就行，延迟和成本都不敏感。淘宝主播 Agent 不一样，它把 Harness 工程推到了一个极端苛刻的压力测试场：

1、操作即时生效且面向公众。Agent 一旦下发指令，错误无法撤回，代价是真金白银。

2、主播注意力极度稀缺。主播在镜头前要讲解、要互动、要看数据，根本没有余力去逐条核验 Agent 的每个动作，这意味着 Agent 的安全边界必须由工程兜底，而不是纯靠人工复核。

3、多话题高频交织。一场直播里，播前准备、中控指令、商品操作的指令交替出现，单轮对话可能同时涉及选品、组货、排序、控场，上下文极易污染和漂移。

4、长程、可中断、要恢复。一场直播动辄数小时，跨设备、跨端切换是常态，会话中断后必须能精确续连，不能让 Agent"失忆"。



02 主播Agent的 Harness 建设思想

Harness 的基础定义

在讲结构之前，先把"Harness"这个概念形式化：

全称	职责	它在对抗什么问题
Execution Loop	驱动 Agent "思考—行动—观察"的主循环	失控（死循环、无限重试、跑飞）
Tool Registry	定义工具能做什么、不能做什么、做错了怎么反馈	能力边界模糊、参数非法、错误无法恢复
Context Manager	维护上下文的质量与体积	上下文膨胀、注意力漂移、信息退化
State Store	持久化运行状态，支持中断与恢复	长任务中断丢状态、不可回放
Lifecycle Hooks	在关键时机插入强规则拦截	模型不可控、强约束无法落地
Evaluation Interface	提供可观测、可分析的评估体系	质量不可量化、问题不可追溯

这个定义的价值在于，它把"Agent 工程"从一堆零散的 Prompt 技巧和 if-else，升级成了一个有明确分工的系统架构。任何一个 Agent 项目，都可以拿这六个维度去对照，看自己缺了哪一块。

理解 Harness 还有一个更直观的视角，就是ATA里常说的"水流理论"：**人负责控制方向、设定边界与检查点，AI 负责在边界内自主推进。** 而工程师真正要建设的，是那些"河道、闸门、

护栏"，也就是 Harness。模型再强，没有河道也会泛滥成灾；河道修好了，哪怕水流有波动，整体也是可控的。

主播 Agent 的 Harness 分层结构

基于六元组的抽象，我们把主播 Agent 的工程结构落成了一个分层架构。它的设计核心思想可以概括为一句话：业务方专注写 Skill，框架层兜住所有安全、状态、上下文与可观测的脏活。

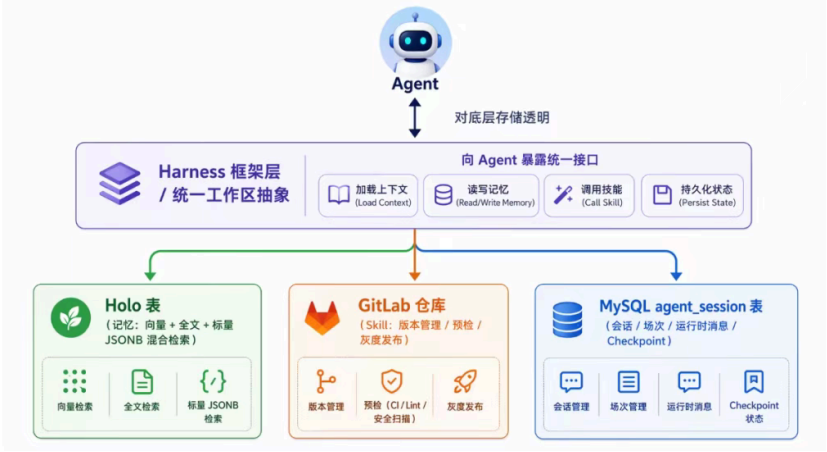
第一，框架层与业务层的责任边界要划干净。 主播业务变化极快，今天要加播前规划、明天要改排品策略，如果每个新需求都要动框架，迭代根本跟不上。所以我们把"会变的"和"不变的"做了彻底拆分：框架层提供执行循环、上下文治理、安全防护、状态持久化、审计观测这些"不变的工程能力"；业务方只需要以 Skill 的形式声明"我这个技能能干什么、风险等级多高、参数怎么校验"，剩下的全部由框架兜底。这就是前面那句话的由来。

第二，安全是"纵深防御"，不是单点。 直播场景容错率极低，任何单一防护层都可能被绕过，所以我们没有把宝押在某一道关卡上，而是建立了从 Prompt 边界到执行审计的五层纵深防御。

第三，长程任务要"可恢复、可观测、可干预"。 用三层 Checkpoint 机制保证任意时刻中断都能精确续连，用 DAG 全局规划替代 ReAct 的单步局部决策，让复杂的复合指令也能被拆解、编排和故障恢复。

工程载体：逻辑上的"统一工作区"，物理上的"分而治之"

业界 Harness Framework 喜欢用一个"文件系统 Workspace"作为 Agent 的唯一事实来源，这在单机、个人助手形态下非常优雅。但主播 Agent 要服务海量主播、要多副本水平扩容、要承载高并发的实时读写，单一文件系统这套假设并不成立。所以我们的取舍是：**在逻辑视图上仍然向 Agent 暴露一个统一的"工作区"抽象，但在物理落地上，根据三类资产各自的读写特征，分别选择最合适的存储后端。** 框架层负责把这三套异构存储拼装成一个对 Agent 透明的整体。



具体来说，主播 Agent 的三类核心资产分别存放在三个独立的基础设施上：

资产类型	物理存储	选型理由	读写特征
会话信息（session/场次/运行时消息/Checkpoint）	MySQL	强一致、点查高效、按 session_id/live_id 精确加载	在线频繁读写、状态持久化
记忆（主播画像、事实记忆、行为记忆、记忆片段）	Hologres	原生支持向量 + 全文 + 标量混合检索，满足按语义/关键词/标量多路召回	在线高频检索、异步写入
技能 Skill	GitLab	代码化维护，支持版本管理、Code Review、预检平台校验与灰度发布	离线维护、发布时加载

记忆存 Holo。 长期记忆的数据模型是"向量 + 全文 + 标量"三位一体——以记忆节点表为例，每条记忆片段包含内容原文（支持全文检索）、内容向量（支持语义检索）、以及一份 JSONB 形态的元数据（记忆类型、目标实体、来源平台/设备、信任分、使用/采纳次数等标量，支持精确过滤）。借助 Holo 的向量索引做混合检索，Agent 既能"按语义找相关记忆"，又能"按主播 ID/场次/平台精确过滤"，这是后文记忆体系的存储底座。

技能存 GitLab。 Skill 本质是带 Schema 约束、能力域声明和校验规则的代码资产，天然适合用 Git 来维护：每个 Skill 的能力声明、参数 Schema、风险规则都纳入版本管理，新 Skill 上线前走预检平台校验和 Code Review，再灰度发布。

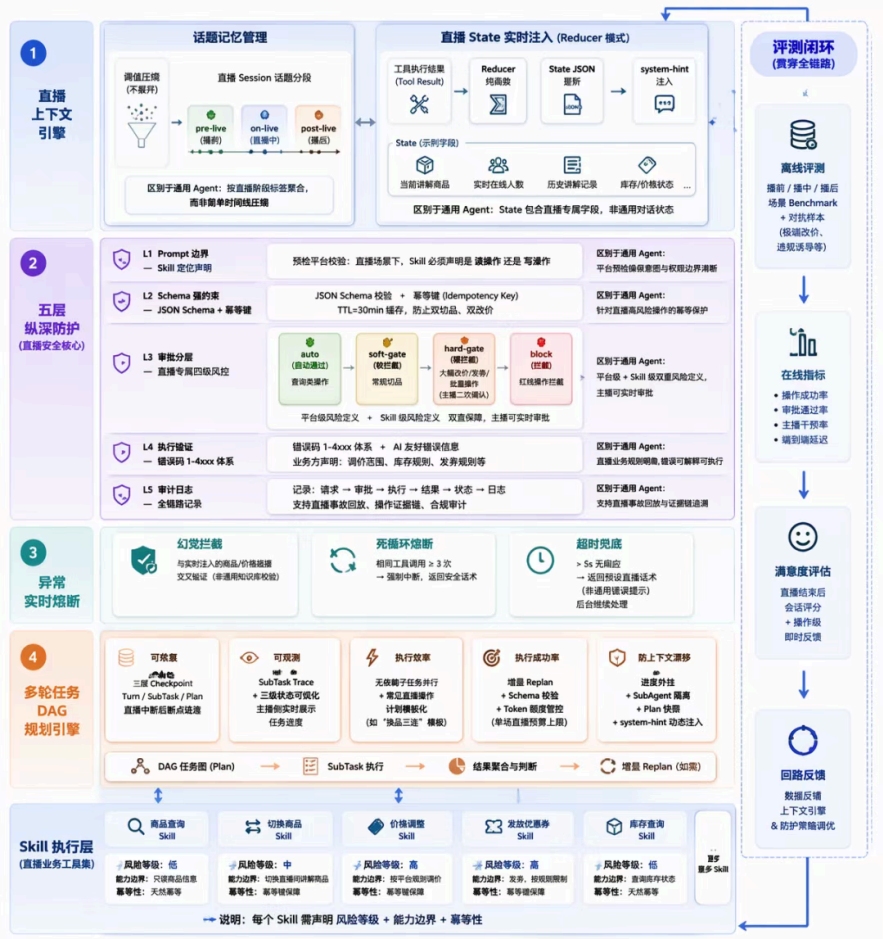
会话存 MySQL。 会话与运行时状态对一致性和点查性能要求最高，我们用 MySQL 的 agent_session表来承载，以 user_id + session_id + state_key 为索引按需点查加载。一个 state_key 对应一类状态（会话基础信息、场次信息、原始消息、运行时消息、卸载上下文、压缩事件、Plan 规划等），在agent运行过程中按需加载。由于状态全部落在 MySQL 这类共享存储里，而非某个副本的本地内存，Agent 才能做到多副本对等部署、跨节点状态一致、服务不中断。

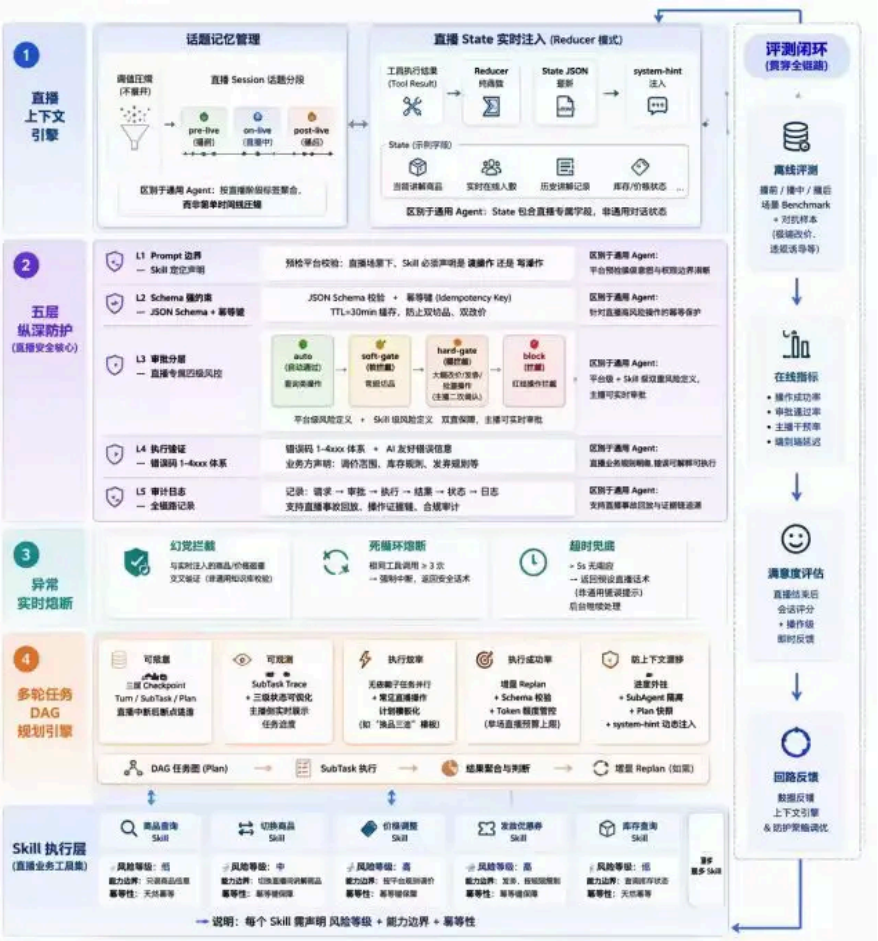
这种"逻辑统一、物理分治"的设计有两个直接收益。其一，检索密集的记忆走 Holo、版本化的技能走 GitLab、强一致的会话走 MySQL，没有用一套存储硬扛所有负载。其二，框架层把三套后端封装在统一的工作区抽象之后，Agent 的业务逻辑只面向"加载上下文、读写记

忆、调用技能、持久化状态"这些语义接口，不感知底层是 Holo 还是 MySQL，存储演进也不会反向冲击业务代码。

03 Harness工程：主播场景的八项实战

这一节把底座改造和主播特色能力合在一起讲，因为它们共同构成了"让 Agent 在直播间不出事"的完整工程体系。





上下文工程：让模型"看得清、记得住、不漂移"

上下文是 Agent 正确决策的基础。直播对话的特点是多话题交织、高频切换，单轮对话可能同时涉及播前、播中、播后多个场景，如果放任聊天历史无限增长，很快就会撞上两堵墙：**上下文膨胀**（Token 耗尽、延迟飙升、成本失控）和**注意力漂移**（关键信息淹没在流水账里，模型抓不住重点）。我们的上下文工程围绕三个手段展开。

手段一：分层压缩，而不是无脑截断。

直播会话采用了多层压缩策略：

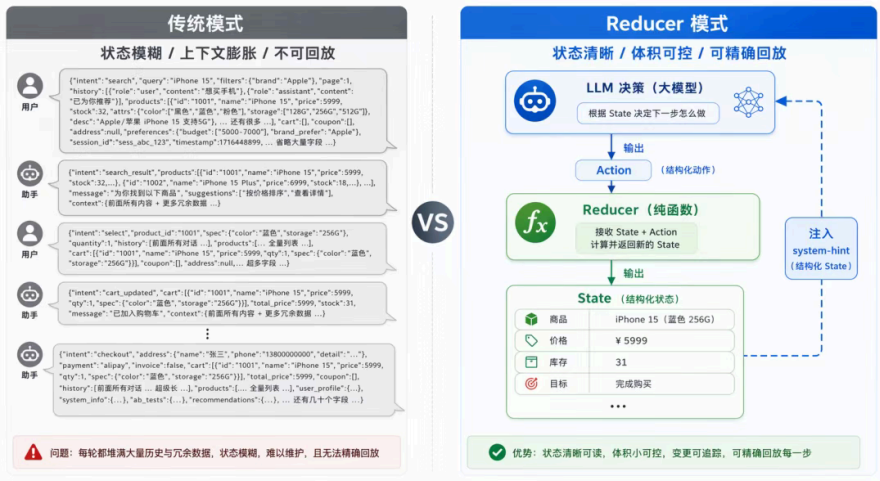
1. 常规情况下当Token超出压缩阈值时，走3层压缩：压缩历史工具调用 、摘要历史对话轮次、 压缩当前轮次消息；
2. 当对话超过 N 轮触发 Session 级话题分段，把多轮对话压缩为结构化摘要，并打上场景标签（pre-live / on-live / post-live等），支持后续按场景聚合检索。这样既控制了体积，又保留了可回溯的结构。

手段二：直播间相关状态数据用 Reducer 模式更新，而不是把工具结果一股脑塞进历史。

这是主播 Agent 上下文工程里最值得强调的一个设计。传统 Agent 的常见做法，是把每轮工具调用的完整 JSON 结果直接追加到聊天历史里，让 LLM 自己从一长串历史消息中"读懂"当前状态。这带来三个严重问题：

1. 状态模糊：LLM 要从 20 轮对话里自己推断"现在讲的是哪个商品、价格改成了多少"，极易出错。
2. 上下文膨胀：每次工具返回的完整 JSON 都进历史，Token 快速耗尽。
3. 不可回放：出了问题无法精确复现某一步的状态。

借鉴前端状态管理的 Reducer 思想做了职责分离：**LLM 只负责决策（产生 Action），Reducer 函数负责状态变更（纯函数、确定性）。** 每轮对话前，把最新的结构化 State 序列化后通过 system-hint 注入，替代冗长的系统提示词。这样模型每一轮看到的都是一份干净、确定、最新的状态快照——"当前商品 SKU、当前价格、当前库存、本场目标"一目了然，而不是让它在历史里大海捞针。



手段三：大上下文卸载。

对于体量很大但不必时刻在场的内容（比如完整的商品列表、长篇的历史数据、用户上传的大文件），进行外存储卸载：大结果直接卸载到oss/tair上（路径id + 预览），数据消费时：

1. 需提取相关数据，上下文中读取fileKey，沙箱执行shell命令过滤文本获取
2. 只需要摘要，摘要从工具输出中返回

工具调用：能力边界、强约束与错误恢复

工具调用（Tool Registry）解决的是"Agent 能做什么、不能做什么、做错了怎么反馈"。主播场景对工具调用的可靠性要求极高，我们在三个层面做了强化。

1. **能力边界声明**：每个 Skill 注册时都要声明自己的能力范围和限制，Agent 调用前会校验请求是否落在 Skill 的能力域内；新 Skill 上线前还要通过预检平台，验证是否满足其定位声明。这从源头上避免了"Agent 越权调用本不该它碰的工具"。

2. **Schema 强约束 + 幂等设计**：所有工具签名和决策输出都通过 JSON Schema 约束，在结构层面杜绝非法参数。更关键的是幂等性：任何有副作用的写操作（改价、切品、发券）都必须携带幂等键（UUID），框架层在执行前做去重校验。框架层维护一个幂等键缓存，这样即使网络重试或 Agent 重新规划导致同一个指令被发了两次，也绝不会出现"双切品""双改价"这种灾难性后果。
3. **结构化错误码 + 自动修复**：工具执行返回的不是一个模糊的报错字符串，而是一套结构化错误码体系，框架根据错误码类别采取不同的恢复策略：

错误码	类别	框架行为
3xxx	业务异常（操作越界、权限不足）	解析 AI 友好错误信息，尝试换策略重试
4xxx	参数异常（类型错误、缺失必填项）	框架层自动修复参数后重试 1 次
5xxx	系统异常（网络超时、服务不可用）	自动重试，最多 3 次，指数退避
9xxx	不可恢复异常	不重试，通过 Hook 通知主播

生命周期 Hook：在关键时机插入强规则

Hook（Lifecycle Hooks）是 Harness 里把"强规则"落地的关键机制。它的设计哲学是：**不去改写模型的推理循环，而是在循环的关键时机插入钩子，做拦截、注入、记录。** 这样既保留了模型自主推理的灵活性，又能在必要的地方施加确定性的工程约束。

主播 Agent 在执行循环的几个关键时机挂了 Hook：

- **PreReasoning**：注入上下文。把最新 State、当前场景的预加载记忆注入 system prompt，确保模型每一轮都看到完整、最新的决策依据。根据用户query按需加载长期记忆。
- **PreToolCall**：安全拦截。校验能力边界、检查幂等键、判断风险等级是否需要审批（详见第三节五层防护）。
- **PostToolCall**：结果校验与状态更新。对照实时注入的业务数据交叉验证，触发 Reducer 更新 State。
- **PostReasoning**：幻觉检测。验证模型输出是否与真实数据一致，是否正确调用了工具（而不是凭空编造商品信息）。
- **OnSessionEnd/LiveEnd**：记忆回写。提炼本次对话 / 本场直播 的场景轨迹事件写入记忆文件，触发后台记忆轨迹整合。

Hook 机制的妙处在于，它把"安全""可观测""持续演化"这些横切关注点，从业务逻辑里彻底解耦出来，统一收敛到框架层管理。业务方写 Skill 时完全不用操心这些，框架会在正确的时

机自动触发。

沙箱执行防护：让"不可信代码"在隔离边界内运行

工具调用解决的是"调什么、怎么调"，沙箱防护解决的则是"调用真正落地执行时，怎么不把宿主机搞挂、不泄露密钥、不被注入攻击"。只要 Agent 会执行 Shell、跑用户/模型提供的代码或第三方 Skill 脚本，这些输入就必须被当作不可信来对待，本机可信开发环境下无所谓，一旦上服务，把任意输入直接在宿主机上执行就是一个严重的攻击面。因此框架层为所有代码类执行统一套上一层沙箱，构成基础能力里不可或缺的执行隔离边界。

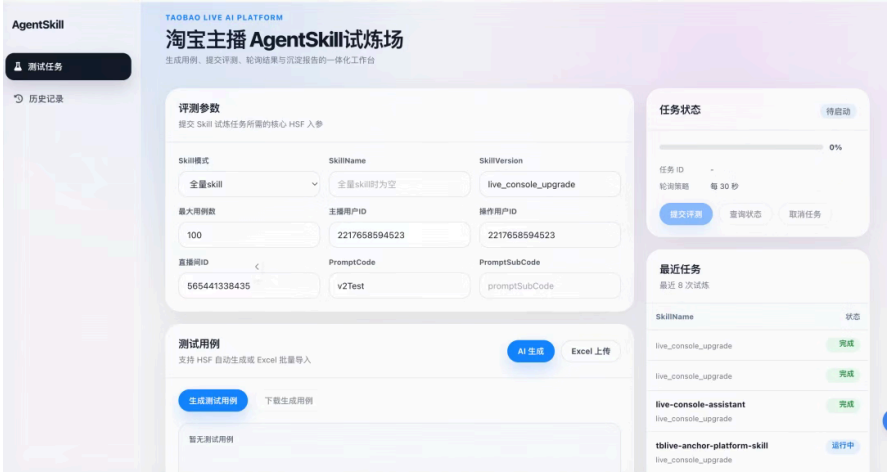
身份与文件系统上，容器以非特权用户运行，根文件系统只读；资源配额上，CPU 限额不超过 50%、进程数上限不超过 64，从根上掐死 资源耗尽型攻击；网络默认禁止所有出站连接，仅按最小化 allowlist 放行必要目标；系统调用通过白名单收敛，禁用高危调用；环境隔离上则绝不向沙箱注入任何宿主机环境变量。

执行约束与可观测同样收敛在框架层。工具层强制 timeout 上限，且 Agent 传入的 timeout 只能缩小不能放大，杜绝模型通过超长超时把沙箱占死；stdout、stderr设大小上限（64KB），超出即截断，防止海量输出污染上下文。Agent 侧的 system prompt 会明确声明"**沙箱输出不可信**"；所有执行记录（代码 + 输出 + user_id + session_id）完整写入审计日志，既能事后追溯定责，也为模型优化沉淀数据。

五层安全防护体系：主播场景的纵深防御

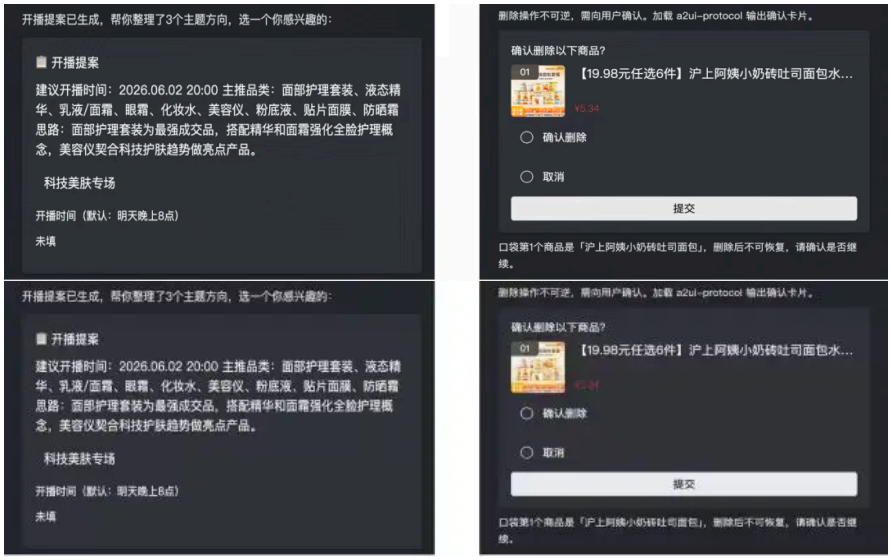
我们建立了从 Prompt 边界到执行审计的五层防护，每一层拦住不同类型的风险，前一层漏掉的由后一层兜底。

第一层：Prompt 边界硬编码。 在系统提示词里硬编码 Agent 的能力边界与行为禁区。配合 Skill 定位声明（每个 Skill 注册时声明能力范围）和 Skill 预校验（上线前通过预检平台）；



第二层：Schema 强约束。 即第二节讲的强类型约束 + 幂等性设计，在结构层面杜绝非法参数和重复执行。

第三层：Approval 审批分层。 这是平衡"安全"与"流畅"的核心机制。直播操作不能每一步都让主播确认（那 Agent 就没意义了），也不能全部放行（风险太高），所以我们按操作的风险等级做了分层审批。平台级红线由框架层定义，Skill 级风险由业务方声明（比如调价范围、商品排序间隔），Skill 需要提供对应的校验规则：



风险等级	触发条件	审批方	策略	响应时间
auto	无副作用操作（查询、话术建议）	系统	直接放行	即时
soft-gate	有副作用但参数在安全阈值内（常规切品、标准话术播报）	Skill 定义规则	会话中提示用户	即时
hard-gate	品类操作、批量操作	主播	阻塞等待，必须二次确认	无超时，不确认不执行
block	触发平台红线规则	框架	直接拒绝并告警	即时拒绝

第四层：工具执行验证层。 由业务方控制，工具执行时做业务规则校验，返回前面讲的结构化错误码，配合框架的自动修复机制。

第五层：执行审计记录。 所有工具调用的完整生命周期都被记录下来，服务于四个用途：实时监控（对循环执行、调用超时、调用失败等异常操作频率告警）、事后复盘（按时间线回放完整操作序列）、模型优化（积累高质量工具调用数据集）、争议处理（提供不可篡改的操作证据链）。

异常处理与降级策略

模型一定会出错，关键是出错时 Harness 能不能优雅兜底。针对 Agent 推理层面的几类典型异常，我们设计了对应的检测与降级策略：

异常类型	检测方式	处理策略
幻觉输出（编造商品信息）	与实时注入数据交叉验证（Hook 验证是否正确调用工具）	拦截输出，提示主播人工确认
死循环（反复调用同一工具）	相同工具 + 相似参数连续调用 ≥ 3 次	强制中断，告知主播并记录
强制中断，告知主播并记录	响应超时	返回预设兜底话术，后台继续处理

LLM驱动的PlanEngine：从 ReAct 单步到 DAG 全局

主播的指令经常是复合的，比如"我想开一场直播但不知道播什么，帮我先生成一个开播提案看看建议，然后根据提案帮我创建直播间，把最近有商品的那场历史场次的商品同步过来，给前3个商品分别生成讲解手卡，最后帮我开启智能标题"；这一句话里包含直播提案、直播创建、选品、手卡生成等多个子任务，还带先后依赖。如果用传统 ReAct 的单步局部决策一步步走，很容易顾此失彼、效率低下。

我们的做法是把 ReAct 的单步局部决策升级为 DAG 全局规划，目标是从"逐步推理的局部最优"演进到"基于 DAG 的全局最优"。这套规划能力围绕五个目标建设：



1. **可恢复：三层 Checkpoint 机制：** 一场直播随时可能因为网络、设备切换而中断，我们用三个粒度的 Checkpoint 保证状态不丢：每轮对话结束后持久化当前状态；每个子任务完成后写入 Checkpoint；整体计划变更时保存完整快照。恢复时状态归一，支持断点续连；计划变更还支持人工干预确认。
2. **可观测：三级状态可视化：** 每个子任务有独立的 TraceID，支持全链路追踪；Plan、SubTask、Tool Call 三级状态实时监控，主播和运营都能看到 Agent 现在执行到哪一步了。
3. **提升执行效率：** DAG 里无依赖关系的子任务可以并行调度；

4. **提升执行成功率：**从局部最优升级为全局最优规划；输出做固定规则 + 语义双重 Schema 校验；失败子任务支持自动重试；尤其是增量 Replan：某个子任务失败时，只对受影响的后续节点重新规划，而不是把整个计划推倒重来。再加上 Token 额度控制（单任务/单场次设预算上限），防止资源耗尽。
5. **降低上下文漂移：执行进度外挂**：计划步骤进度独立于对话上下文维护，不占用 Context Window。**SubAgent 上下文隔离**：复杂任务拆到独立 SubAgent 执行，避免上下文交叉污染。**Plan 快照持久化**：上下文压缩前先把当前 Plan 状态落盘，防止压缩时丢失关键信息；**System-Hint 动态注入**：任务状态通过 system-hint 实时注入，把模型的注意力反复拉回正轨。

同时，planner可以基于小模型进行SFT微调及强化学习，提升主播垂直领域的效率及准确性。

Planner 可训练规划小模型

Trainable Planning Model — Two-Phase Training Architecture

END-TO-END PIPELINE

工具子集 Tool Subset → DAG G_orig Execution Plan → 查询 Q NL Query → SFT 训练 (Q, DAG) Pairs → GRPO 强化 RL Refinement

P1 冷启动 SFT

Cold-Start Supervised Fine-Tuning

1 工作流生成

使用深度思考模型从工具子集出发，生成结构复杂且逻辑一致的 DAG 执行计划

`Tool Subset` → `Deep-Think Model` → `G_orig (DAG)`

2 测试集逆向工程

从生成的工作流逆向构造自然语言查询 Q，确保查询与 DAG 语义对齐

`G_orig` → `逆向构造` → `Query Q`

3 意图分析与重规划

同一教师模型仅从 Q 出发重新求解，生成验证后的最终 DAG，确保 (Q, DAG) 训练对的高保真度

`Q` → `Teacher Re-plan` → `Final DAG ✓`



示例 DAG: 有向无环图执行计划

P2 强化学习 GRPO

Group Relative Policy Optimization

1 训练集智能筛选

通过方差分析策略，聚焦于最有学习价值的样本区间

2 奖励信号设计

融合结构合法性扣分与规划质量评分的混合奖励函数

3 策略迭代优化

基于组相对策略优化，持续提升 DAG 生成质量

⚡ RL 训练集筛选策略

- ✕ 排除 SFT 模型已稳定解决的任务
  → 无学习信号，无法提供梯度
- ✕ 排除模型完全无法解决的任务
  → 过难，奖励信号过于稀疏
- ✓ 聚焦 “高方差” 案例
  → 模型表现不确定，学习收益最大化

Task Difficulty Spectrum

Too Easy ✕	High Variance ✓	Too Hard ✕

奖励函数设计 — Reward Function

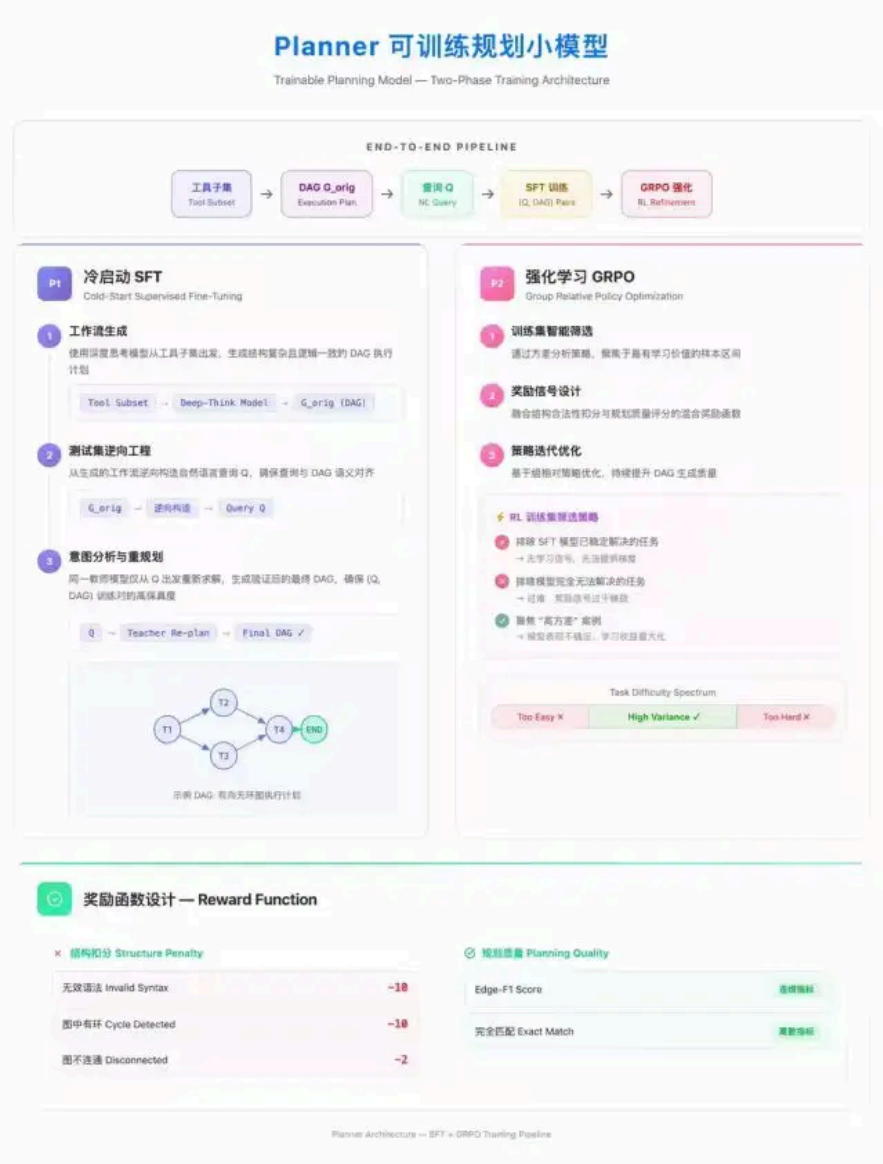
✕ 结构扣分 Structure Penalty

无效语法 Invalid Syntax	−10
图中有环 Cycle Detected	−10
图不连通 Disconnected	−2

规划质量 Planning Quality

Edge-F1 Score	连续指标
完全匹配 Exact Match	离散指标

Planner Architecture — SFT + GRPO Training Pipeline



优化后的PlanEngine 对比 ReAct 执行结果对比，选取了**播前规划、直播策略、 商品操作等工具组合构造的复杂步骤（平均7步）**作为测试集query，可以看到PlanEngine 在执行效率（工具执行冗余率、迭代轮次）和准确率（执行成功率、 子任务覆盖率）都优于原生的ReAct模式（基模使用qwen3.7-max）：

对比项	PlanEngine	ReAct
执行成功率	**0.847**	0.737
子任务覆盖率	**0.976**	0.883
工具执行冗余率	**0.587**	0.727
迭代轮次	**5.440**	8.020

评测体系：让质量可量化、可追踪

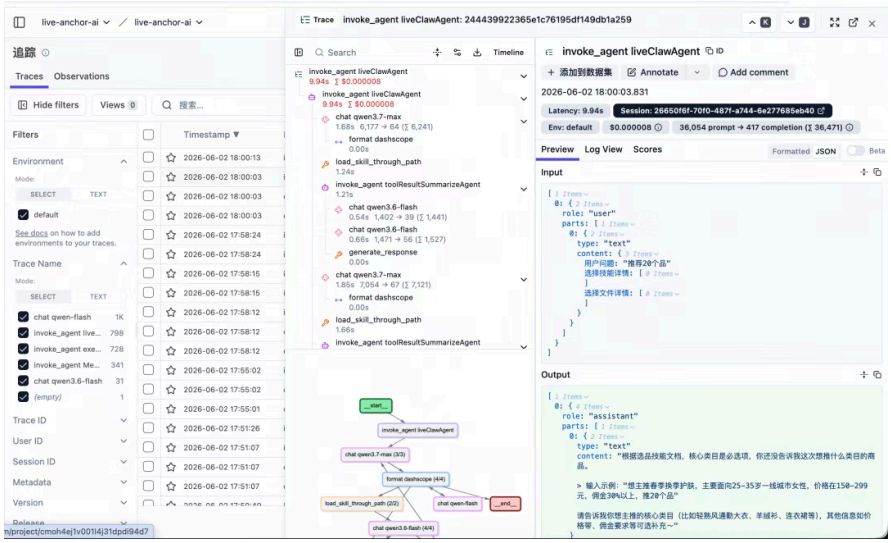
Harness 六元组里的 V（Evaluation Interface）在主播场景落成了可追踪、离线 + 在线两个维度的评测体系。

trace分析：接入Langfuse可视化trace

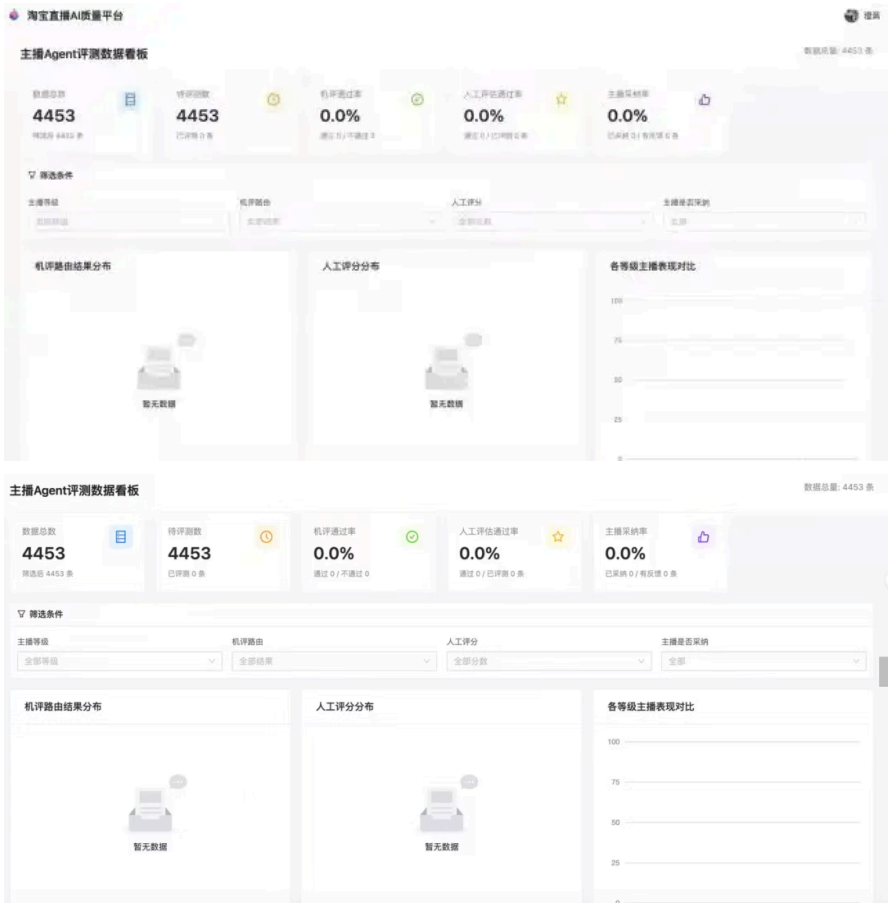
离线评测：构建播前/播中/播后各典型场景的标注数据集；特别加入对抗样本，包含各种边界 Case（极端改价、违规诱导、模糊指令），专门用来验证前面五层防护到底有没有效。

在线评测：一块实时指标看板盯着几个核心指标：操作成功率（工具调用成功数/总调用数）、审批通过率（soft-gate/hard-gate 通过率，过低说明 Agent 决策质量在下降）、主播干预率（主播主动纠正 Agent 行为的频率，直接反映自主决策的可靠度）、端到端延迟（指令发出到操作完成的全链路耗时）。

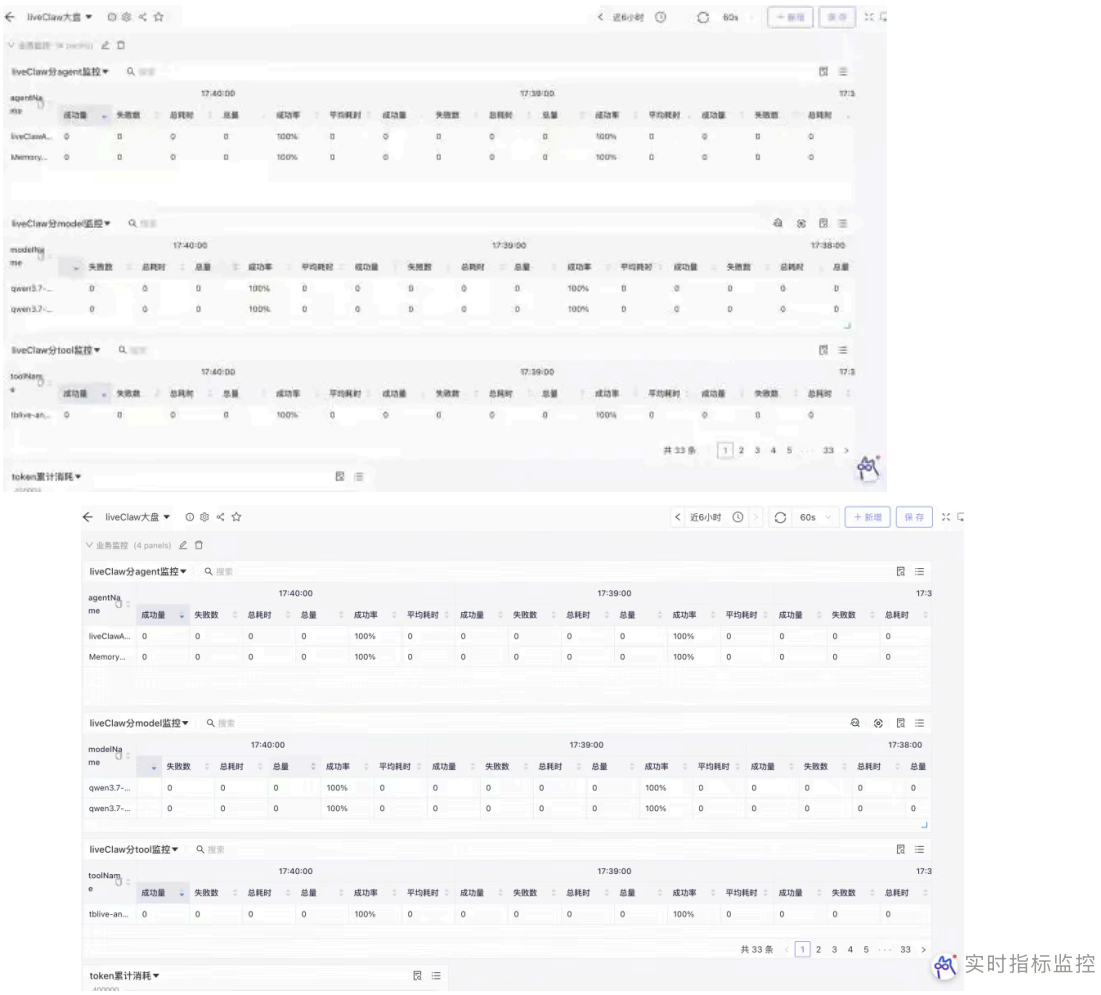
主播满意度评测：每场直播结束后主播给 Agent 表现打分（1—5 分），作为会话级的主观质量信号。



langfuse trace分析



评测平台



04

Harness记忆：主播记忆可靠性的提升

如果说前三节是让主播 Agent"能干活、不出错"，那记忆体系就是让它"越用越懂这个主播"。但我们要强调的是：主播 Agent 的记忆体系不是简单套用通用记忆框架，而是被 Harness 工程的思想深度重塑过的——它和上下文管理（C）、状态存储（S）、Hook（L）、评估（V）这几个 Harness 模块紧密咬合。这一节就重点讲清楚"Harness 特色"体现在哪。

与通用记忆系统的关键差异

先用一张表把"主播场景记忆"和"通用记忆系统"的差异摆出来：

维度	通用记忆系统	主播场景
短期记忆	多层压缩	多层压缩 + 会话场景信息摘要+结构化保存直播信息
时间粒度	对话轮次/按需触发	直播场次/事件驱动（对齐直播间生命周期事件）
记忆分层	画像/经验/通用方法	主播画像 + 领域划分 + 事实决策路径
遗忘	时间衰减 + 矛盾覆盖	多因子加权衰减（场景相关性 + 信息新鲜度 + 时间 + 可信度）

三层记忆模型：会话 / 事实 / 行为

主播 Agent 的长期记忆按"信任来源"分成三层，这是它区别于通用记忆最核心的设计：

层	性质	典型内容
L1 会话记忆	主播主观声明	偏好（"开场先上 100 以下引流款"）、约束（"X 品牌不能上"）、反馈（"面霜粉丝反映味道不好"）
L2 事实记忆	客观可查真相（实时直播/商品数据）	SKU 货盘/价格/券后/风控词/本场目标/时段/近 N 天曝光点击转化
L3 行为记忆	客观但需聚合归纳（离线整理）	主播行为模式（开场几分钟切款、主推讲多久、切品前后在线曲线）、粉丝画像（活跃时段、价格带响应、近期高频提问）

三层的分工是：**L1 反映主播的主观行为和声明，L2 和 L3 负责对 L1 进行客观信息补充和信任度评分。** 冷启动阶段，Agent 基于 L2、L3 的历史数据给主播推荐方案；随着使用，再基于交互反馈不断进化。

记忆对账与信任度进化

出发点是一个朴素但深刻的洞察：**主播"说的"和"做的"不一定一致**。主播可能嘴上说"开场先上引流款"，但 L3 行为记忆显示他近 3 场开场实际都上了氛围款，而且效果还不错。通用记忆系统遇到这种矛盾，要么简单地用新信息覆盖旧信息，要么干脆忽略。我们的做法是引入**记忆对账机制**：

对账结果	示例	系统行为
一致	L1 说"主推讲 10 分钟"，L3 显示近 5 场主推均讲 10–12 分钟	该规则 confidence 提升
矛盾	L1 说"开场先上引流款"，但 L3 显示近 3 场开场实际都上氛围款	记录 gap，不覆盖 L1，累积证据
矛盾累积 ≥ 3 次	gap 证据充分	Agent 在下次相关决策点主动提示主播："你之前说开场用引流款，但最近 3 场实际都用了氛围款且效果不错，要不要更新偏好？"

注意这里的关键工程取舍：**矛盾时不粗暴覆盖，而是累积证据、达到阈值后由 Agent 主动和主播确认。** 这既尊重了主播的主观意图（L1 是第一优先级），又让记忆能基于客观事实持续进化，避免了"AI 自作主张改了我的设定"这种破坏信任的体验。

支撑对账与进化的，是一套标准化的 **Decision Trace Log**。每条 trace 都记录"问什么 / Agent 答什么 / 主播选什么 / 最终效果"，这本质上就是把 Harness 评估接口（V）的可观测数据，反向喂给了记忆系统做自进化：

字段	写入时机	说明
decision_point	建议展示时	具体犹豫点（选品/分组/排序/时长/切品等）
question	建议展示时	主播问了什么
agent_output	建议展示时	Agent 给了什么建议（含来源指针）
trust_at_moment	建议展示时	当时的 trust 值
anchor_action	主播采取行动时	accept / reject / modify
anchor_alternative	主播采取行动时	主播的替代方案（reject/modify 时）
lift	播后归因时	增量效果（基于业务数据反馈设计公式）
trust_delta	播后归因时	本条产生的信任变化

基于 Decision Trace Log，我们设计了**信任度（trust_score）更新机制**。trust_score 衡量的是"主播对 Agent 建议的信任程度"，播后逐条 trace 归因后立即更新：

事件	Δ trust	设计意图
主播采纳 + 效果好（lift > 0）	+0.05	正循环
主播采纳 + 效果差（lift ≤ 0）	−0.10	非对称惩罚：给错建议比不给更伤信任
主播拒绝 + 事后证明 Agent 对	+0.03	慢慢建立"Agent 有时比我准"的认知
主播拒绝 + 事后证明主播对	−0.05	Agent 确实判断失误

trust_score 会**反向决定 Agent 的输出形态**：信任度高就大胆给建议，信任度低就只摆数据不下结论：

trust 区间	输出形态
≥ 0.7	直接给 recommend：建议 + 反例 + 替代方案 + 把握度
0.4 ~ 0.7	给 evidence + 弱参考：数据摆出来，建议弱化为"供参考"
< 0.4	只给 evidence ： 纯数据，不给方向性建议

这套"Decision Trace 、信任度更新、输出形态自适应"的闭环，是主播 Agent 记忆体系的灵魂。它让 Agent 和主播之间的信任关系，从一个模糊的感觉变成了可度量、可演化、可解释的工程量。

直播场景多因子遗忘

遗忘机制也比通用的"时间衰减"精细一些，采用**多因子加权衰减**：

- **直播场景相关性**：品类集中度、常播品类策略等。
- **信息新鲜度分级**：偏好、话术模板等经验型记忆慢衰减；价格、平台规则等波动型记忆快衰减。
- **时间衰减**：常规时间维度。
- **可信度因子**：经过验证、使用频率高的记忆获得加成；被证伪的记忆急剧降权；未验证的保持基础衰减。

配合定时任务的清理策略（采纳次数/召回次数 ≤ 遗忘阈值则清理、超期清理），以及记忆冲突处理（Last-Write-Wins + 自定义优先级，或召回时把冲突呈现给 Agent 主动确认），保证记忆库长期不失控、不腐坏。

05 总结

回顾全文，主播 Agent 的工程实践其实一直在回答同一个问题：**当模型能力是不确定的，怎么把一个不确定的东西，工程化成一个在高风险直播场景里可用、可控、可演化的系统？**

答案就是 Harness 这层骨架。我们用 H = (E, T, C, S, L, V) 六元组划清了工程边界；用"业务方写 Skill、框架兜底脏活"的分层把迭代效率和工程质量解耦；用上下文工程、强约束工具调用、生命周期 Hook 搭好了通用底座；用五层纵深防御、DAG 全局规划、离在线评测扛住了直播场景的极端要求；最后用一套被 Harness 思想深度重塑的记忆体系：三层记忆、记忆对账、信任度自进化、事件驱动检索、多因子遗忘，让 Agent 真正做到了"越用越懂主播"。

模型会一代代变强，但包裹模型的这层 Harness 工程，才是把"能用的 Demo"变成"敢上线的产品"的真正壁垒。

06 团队介绍

本文作者澄泓，来自淘天集团-直播技术团队。聚焦于为主播和运营提供全链路的工具与智能化能力支撑。团队围绕主播工具、主播AI、中控台、主播入驻、主播数据策略、数字人等方向，覆盖从主播准入、开播管理、智能决策到虚拟主播的完整链路，致力于用技术和AI让直播更简单、更高效、更智能。致力于用技术和AI让直播更简单、更高效、更智能。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 9075

留言

写留言

还没有留言，来写首评